



UNIVERSITY OF BARISHAL
**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING**

Lab Report

Course Title: Artificial Intelligence Lab
Course Code: CSE-3206

Submitted By

Name: SOURAV DEBNATH
Roll: 22CSE009

Submitted To

Dr. Tania Islam
Assistant Professors
Department of CSE
University of Barishal

Report Overview

This report documents the HealthPlus Hospital Heart Disease Prediction project on a task-by-task basis, strictly following the structure of the executed Google Colab notebook (Sourav_debnath_22CSE009. ipynb) submitted for the Artificial Intelligence Final Lab Examination.

For every task, this report states the exact AI prompt that was used to generate or guide that section of the notebook, explains why that prompt was chosen, reproduces the exact Python code from the notebook together with a brief line-by-line explanation, reports the console output exactly as produced, and presents every table and figure with a caption, a description, and an interpretation. Each task section closes with a discussion, 3-5 key findings, and a task-level conclusion.

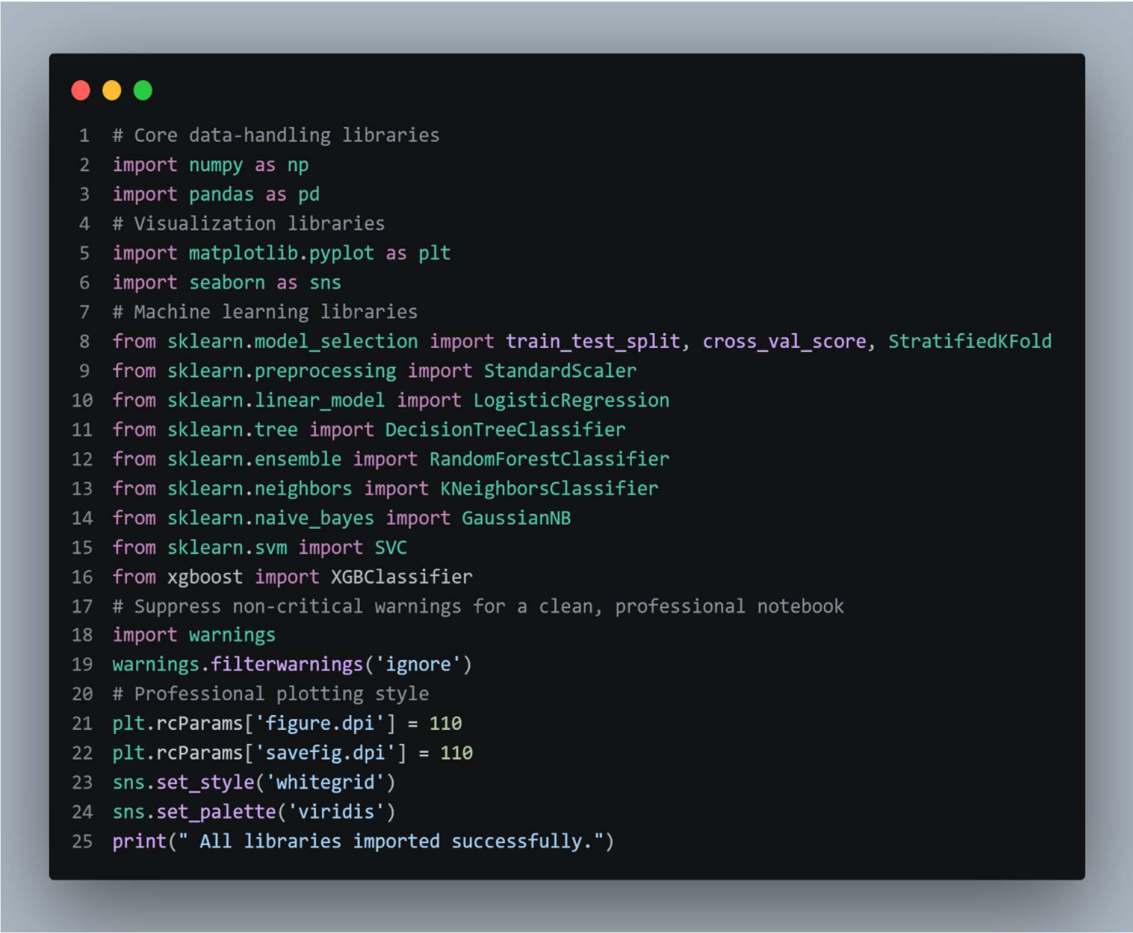
All numerical results, tables, and figures in this report are extracted directly from the executed notebook outputs. No results have been invented, estimated, or altered; where the notebook's own code produced a particular number (for example, a 70.5% duplicate-row rate or an 89.83% ROC-AUC for Naive Bayes), that exact number is reproduced here.

Dataset Description

The dataset used in this project, heart.csv, contains 1,025 patient records with 13 input (clinical/demographic) attributes and one binary target label. It is a widely used, publicly available heart-disease dataset in the style of the UCI Cleveland Heart Disease dataset. The notebook begins by importing the required libraries and loading the dataset, before inspecting its first rows, last rows, and a random sample to confirm correct loading.

Import Libraries and Load Dataset

Python Code:



```
1 # Core data-handling libraries
2 import numpy as np
3 import pandas as pd
4 # Visualization libraries
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 # Machine learning libraries
8 from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.ensemble import RandomForestClassifier
13 from sklearn.neighbors import KNeighborsClassifier
14 from sklearn.naive_bayes import GaussianNB
15 from sklearn.svm import SVC
16 from xgboost import XGBClassifier
17 # Suppress non-critical warnings for a clean, professional notebook
18 import warnings
19 warnings.filterwarnings('ignore')
20 # Professional plotting style
21 plt.rcParams['figure.dpi'] = 110
22 plt.rcParams['savefig.dpi'] = 110
23 sns.set_style('whitegrid')
24 sns.set_palette('viridis')
25 print(" All libraries imported successfully.")
```

- numpy and pandas provide array and DataFrame operations for numerical/tabular data handling.
- matplotlib and seaborn provide the plotting engine used for every chart in the notebook.
- sklearn.model_selection supplies train_test_split and cross-validation utilities.

- `sklearn.preprocessing.StandardScaler` is imported for later feature scaling.
- Six classifier classes are imported: `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, `KNeighborsClassifier`, `GaussianNB` (Naive Bayes), and `SVC` (imported but not used in the final six-model comparison), plus `XGBClassifier` from the `xgboost` package.
- `sklearn.metrics` supplies every evaluation metric used later: accuracy, precision, recall, F1, ROC-AUC, ROC curve, and confusion matrix.
- `warnings.filterwarnings('ignore')` suppresses non-critical library version warnings so notebook output stays clean.
- `plt.rcParams` and `sns.set_style/set_palette` configure a consistent, professional visual style (110 DPI, whitegrid background, viridis colour palette) used throughout the notebook.

Console Output:

All libraries imported successfully.

Python Code (Load Dataset):

```
1 # Load the dataset
2 df = pd.read_csv('heart.csv')
3
4 # Basic inspection
5 print("Dataset shape (rows, columns):", df.shape)
6 print("\nColumn names:", list(df.columns))
```

Console Output:

Dataset shape (rows, columns): (1025, 14)

Column names: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']

Table 0.1: Dataset Attribute Dictionary

Column	Description	Type
age	Patient age in years	Numerical
sex	Sex (1 = male, 0 = female)	Categorical
cp	Chest pain type (0–3)	Categorical
trestbps	Resting blood pressure (mmHg)	Numerical
chol	Serum cholesterol (mg/dl)	Numerical
fbs	Fasting blood sugar > 120 mg/dl (1 = true)	Categorical
restecg	Resting ECG results (0–2)	Categorical
thalach	Maximum heart rate achieved	Numerical
exang	Exercise-induced angina (1 = yes)	Categorical
oldpeak	ST depression induced by exercise	Numerical
slope	Slope of peak exercise ST segment (0–2)	Categorical
ca	Number of major vessels coloured by fluoroscopy (0–4)	Categorical
thal	Thalassemia test result (0–3)	Categorical
target	Heart disease presence (1 = disease, 0 = no disease)	Target

This table lists every column in heart.csv, its clinical meaning, and its data type, forming the basis for the feature-type separation performed in Task 1.

Head, Tail, and Sample Views

Python Code:



Console Output:

	age	sex	cp	trestbps	chol	fb	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

The `tail()` and `sample(5, random state=42)` cells (executed identically in the notebook) additionally confirmed that the end of the file is not truncated and that a random cross-section of rows looks clinically plausible, with no ordering bias by class.

Task 1: Dataset Understanding

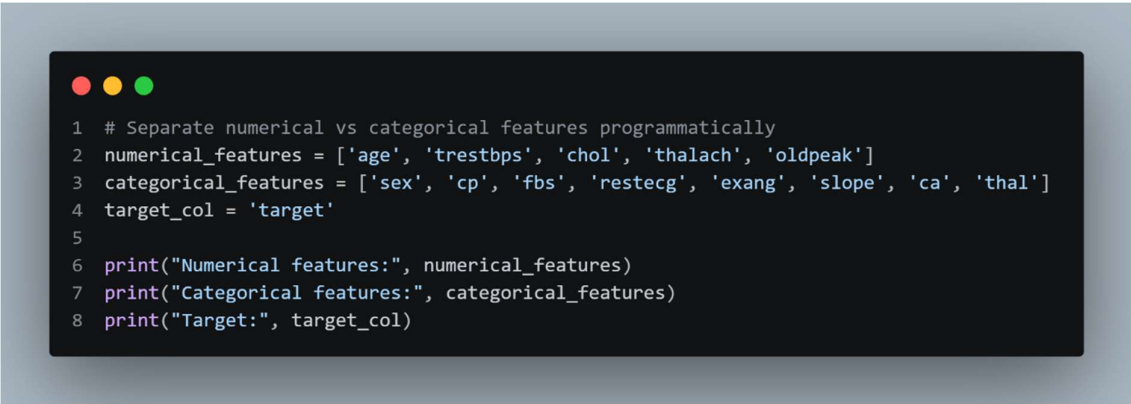
Objective

To identify the prediction target, correctly classify the problem type, separate the input columns into numerical and categorical features, shortlist the clinically most important features, identify key challenges in the dataset, and explain the role of AI/ML in solving this problem.

AI Prompt Used

"Identify the target variable in heart.csv and state whether this is a classification or regression problem, with justification. Separate the columns into numerical and categorical features, suggest which features are likely most important from a clinical standpoint, list two key challenges in the dataset, and explain how AI/ML helps solve this kind of problem."

Python Code



```
1 # Separate numerical vs categorical features programmatically
2 numerical_features = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
3 categorical_features = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
4 target_col = 'target'
5
6 print("Numerical features:", numerical_features)
7 print("Categorical features:", categorical_features)
8 print("Target:", target_col)
```

Code Explanation

- `numerical_features` is a hard-coded list of the five continuous-valued clinical measurements in the dataset.
- `categorical_features` lists the eight columns that represent discrete clinical categories or codes.
- `target_col` stores the name of the label column ('target') for reuse throughout the notebook.
- The three print statements confirm the assignment by displaying each list to the console.

Console Output

Numerical features: ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

Categorical features: ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']

Target: target

Results

- **Target variable:** target (0 = no disease, 1 = disease present).
- **Problem type:** Binary classification the target has exactly two discrete outcome classes, not a continuous numeric value, so classification (not regression) is the correct framing.
- **Numerical features (5):** age, trestbps, chol, thalach, oldpeak.
- **Categorical features (8):** sex, cp, fbs, restecg, exang, slope, ca, thal.
- **Clinically important features shortlisted:** cp (chest pain type), thalach (maximum heart rate), oldpeak/slope (ST-segment response), ca (vessel count), and thal (thalassemia result) these are the classic indicators cardiologists rely on.
- **Two key challenges identified:** (i) a small sample size (1,025 rows, later found to include many duplicates) relative to the complexity of the prediction problem, increasing overfitting risk; and (ii) categorical variables stored as integer codes, which risks being misinterpreted as ordinal/continuous by certain models if not handled carefully.
- **Role of AI/ML:** machine learning can automatically discover the combination of risk factors that best separate diseased from healthy patients a multivariate pattern that is difficult for a human to assess by eye across 13 variables simultaneously and can screen new patients in milliseconds once trained.

Task 2: Exploratory Data Analysis (EDA)

Objective

To numerically and visually explore the dataset, its shape, data types, missing values, duplicates, statistics, unique category values, and class balance and to produce a set of meaningful visualizations that reveal relationships between clinical features and heart-disease status.

AI Prompt Used

"Perform a complete exploratory data analysis on the heart disease dataset: show shape, data types, missing values, duplicates, statistical summary, unique values, and class distribution. Then create at least 6 meaningful visualizations (histogram, boxplot, countplot, scatterplot, heatmap, violin/pie chart), each with a stated purpose and interpretation."

Dataset Structure Checks

Python Code:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It contains four lines of Python code for checking dataset dimensions and data types.

```
1 # Dataset dimensions and data types
2 print("Shape:", df.shape)
3 print()
4 df.info()
```

Console Output:

```

Shape: (1025, 14)

RangeIndex: 1025 entries, 0 to 1024

Data columns (total 14 columns):
#   Column    Non-Null Count  Dtype
---  ---
0   age       1025 non-null   int64
1   sex       1025 non-null   int64
2   cp        1025 non-null   int64
3   trestbps  1025 non-null   int64
4   chol      1025 non-null   int64
5   fbs       1025 non-null   int64
6   restecg   1025 non-null   int64
7   thalach   1025 non-null   int64
8   exang     1025 non-null   int64
9   oldpeak   1025 non-null   float64
10  slope     1025 non-null   int64
11  ca        1025 non-null   int64
12  thal      1025 non-null   int64
13  target    1025 non-null   int64

dtypes: float64(1), int64(13)

memory usage: 112.2 KB

```

Python Code (Missing Values):

```
1 # Missing values check
2 missing = df.isnull().sum()
3 print("Missing values per column:\n", missing)
4 print("\nTotal missing values:", missing.sum())
```

Console Output:

Total missing values: 0 (0 for every one of the 14 columns)

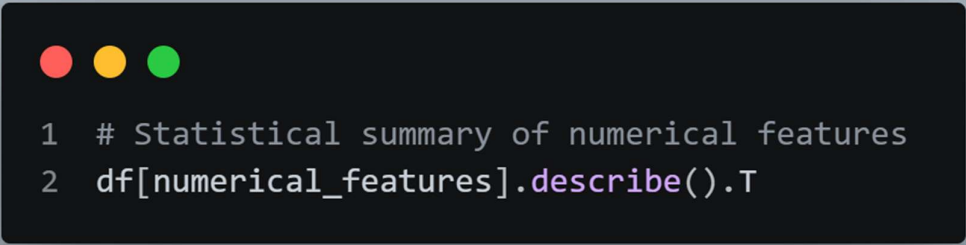
Python Code (Duplicates):

```
1 # Duplicate rows check
2 n_duplicates = df.duplicated().sum()
3 print("Number of exact duplicate rows:", n_duplicates)
4 print(f"Percentage of dataset that is duplicated: {n_duplicates/len(df)*100:.1f}%")
```

Console Output:

Number of exact duplicate rows: 723

Percentage of dataset that is duplicated: 70.5%

Python Code (Statistical Summary):


```

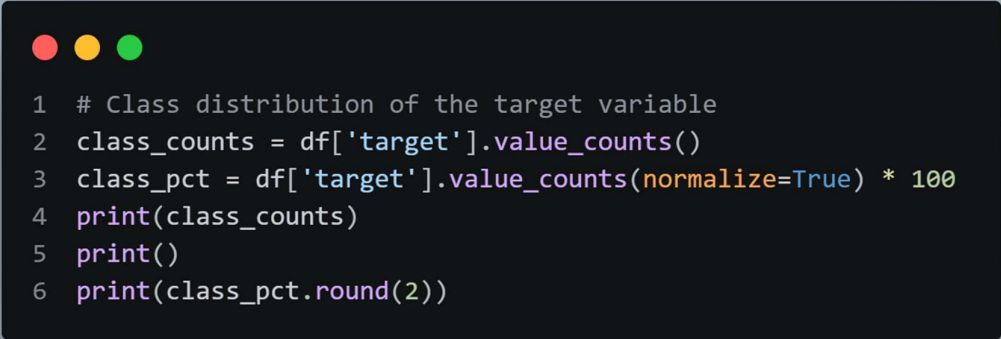
1 # Statistical summary of numerical features
2 df[numerical_features].describe().T

```

Table 2.1: Statistical Summary of Numerical Features

Feature	Count	Mean	Std	Min	25%	50%	75%	Max
age	1025	54.43	9.07	29	48	56	61	77
trestbps	1025	131.61	17.52	94	120	130	140	200
chol	1025	246.00	51.59	126	211	240	275	564
thalach	1025	149.11	23.01	71	132	152	166	202
oldpeak	1025	1.07	1.18	0.0	0.0	0.8	1.8	6.2

This table gives the descriptive statistics (count, mean, standard deviation, and quartiles) for the five numerical features, providing the baseline distribution understanding used later for outlier treatment (Task 3).

Python Code (Class Distribution):


```

1 # Class distribution of the target variable
2 class_counts = df['target'].value_counts()
3 class_pct = df['target'].value_counts(normalize=True) * 100
4 print(class_counts)
5 print()
6 print(class_pct.round(2))

```

Console Output:

```

target
1    526
0    499

Name: count, dtype: int64

target
1    51.32
0    48.68

Name: proportion, dtype: float64

```

Table 2.2: Dataset Summary Checks

Check	Result
Shape	1025 rows × 14 columns
Missing values	0 (across all columns)
Duplicate rows	723 (70.5% of the dataset)
Class balance	Target = 1 (disease): 526 (51.32%) Target = 0 (no disease): 499 (48.68%)
Mean age	54.4 years (range 29–77)
Mean cholesterol	246.0 mg/dl (range 126–564)
Mean resting BP	131.6 mmHg (range 94–200)

Consolidated summary of every structural check performed on the dataset before visualization.

Visualizations

Seven distinct, professionally styled visualizations were produced, exceeding the minimum requirement of six chart types, each covering a different combination of features and a different chart family (heatmap, histogram, boxplot, countplot, scatterplot, pie chart, and violin plot).

Python Code (Correlation Heatmap):

```
1 # Correlation matrix heatmap
2 plt.figure(figsize=(11, 9))
3 corr = df.corr()
4 sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
5             linewidths=0.5, cbar_kws={'label': 'Correlation coefficient'})
6 plt.title('Correlation Matrix of All Clinical Features', fontsize=14, fontweight='bold')
7 plt.tight_layout()
8 plt.savefig('viz_01_correlation_heatmap.png')
9 plt.show()
```

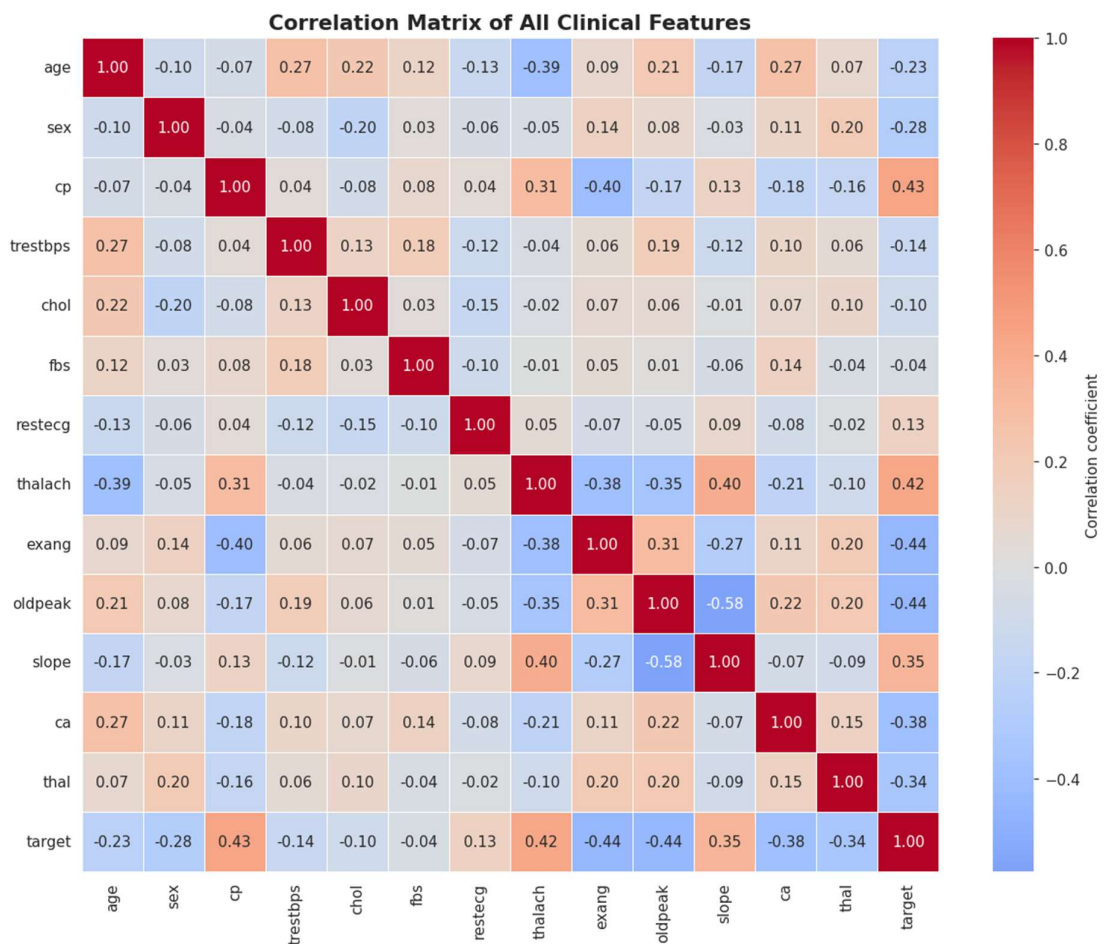


Figure 2.1: Correlation matrix of all 14 clinical features.

Purpose: To reveal linear relationships between every pair of variables, especially between each feature and the target.

Interpretation: cp, thalach, and slope correlate positively with disease status (target); exang, oldpeak, ca, and thal correlate negatively matching the clinical shortlist proposed in Task 1.

Python Code (Age Histogram):

```
1 # Visualization 2: Age distribution (Histogram)
2 plt.figure(figsize=(8, 5))
3 sns.histplot(df['age'], bins=20, kde=True, color='#4C72B0')
4 plt.title('Distribution of Patient Age', fontsize=13, fontweight='bold')
5 plt.xlabel('Age (years)')
6 plt.ylabel('Number of Patients')
7 plt.grid(alpha=0.3)
8 plt.tight_layout()
9 plt.savefig('viz_02_age_histogram.png')
10 plt.show()
```

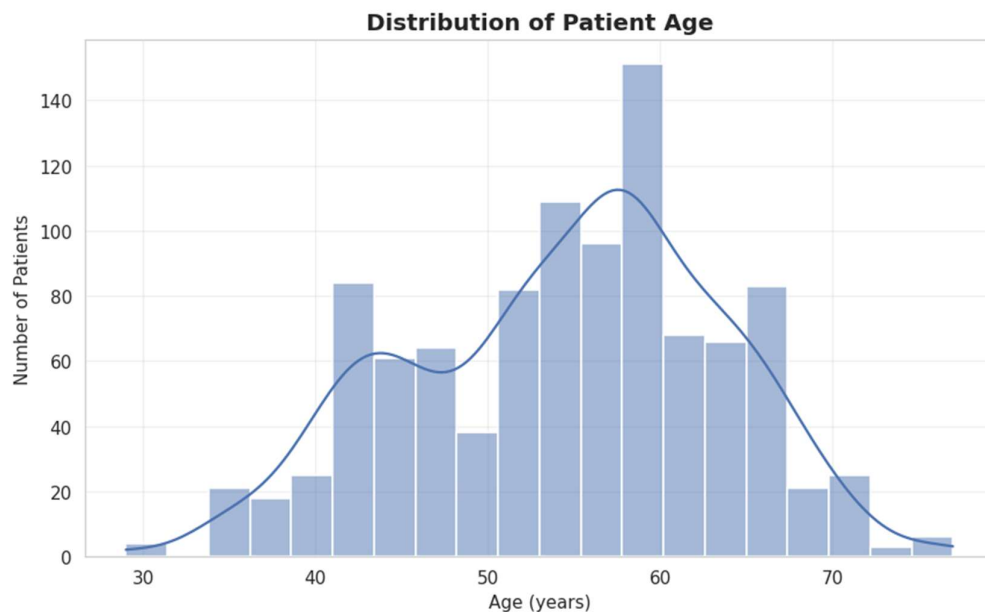


Figure 2.2: Distribution of patient age, with kernel density estimate overlay.

Purpose: To understand the age profile of the patient population sampled for this study.

Interpretation: Ages roughly follow a bell shape centred around 50–60 years, with very few patients under 35, consistent with heart disease being predominantly an older-adult condition.

Python Code (Chest Pain Countplot):

```

1 # Visualization 3: Chest pain type vs target (Countplot)
2 plt.figure(figsize=(7, 5))
3 sns.countplot(data=df, x='cp', hue='target', palette='viridis')
4 plt.title('Chest Pain Type vs Heart Disease Status', fontsize=13, fontweight='bold')
5 plt.xlabel('Chest Pain Type (0-3)')
6 plt.ylabel('Number of Patients')
7 plt.legend(title='Target', labels=['No Disease', 'Disease'])
8 plt.tight_layout()
9 plt.savefig('viz_04_chestpain_countplot.png')
10 plt.show()

```

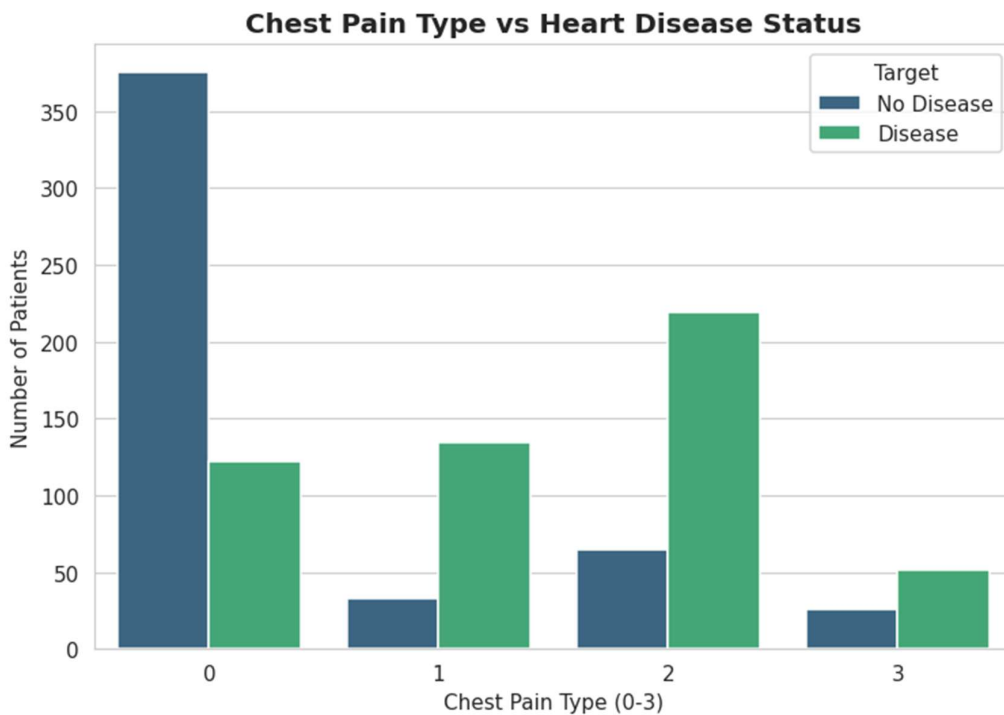


Figure 2.3: Count of patients by chest pain type (0–3), split by disease status.

Purpose: To examine whether chest-pain type distinguishes diseased from healthy patients.

Interpretation: Type 0 is dominated by 'no disease' patients, while types 1–3 are dominated by 'disease' patients a clear, clinically sensible signal.

Python Code (Age vs Heart Rate Scatterplot):

```

1 # Visualization 4: Age vs Max Heart Rate (Scatterplot)
2 plt.figure(figsize=(7, 5))
3 sns.scatterplot(data=df, x='age', y='thalach', hue='target', palette='viridis', alpha=0.7)
4 plt.title('Age vs Maximum Heart Rate Achieved, coloured by Disease Status', fontsize=12, fontweight='bold')
5 plt.xlabel('Age (years)')
6 plt.ylabel('Max Heart Rate Achieved (thalach)')
7 plt.legend(title='Target', labels=['No Disease', 'Disease'])
8 plt.tight_layout()
9 plt.savefig('viz_05_age_thalach_scatter.png')
10 plt.show()

```

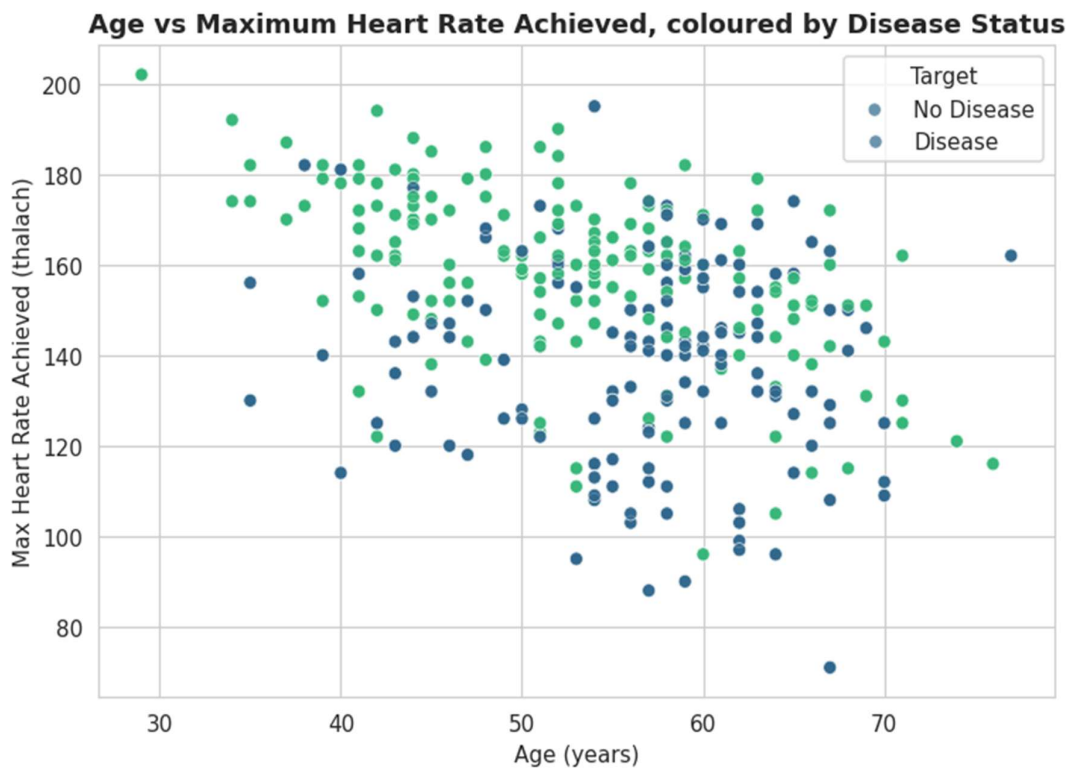


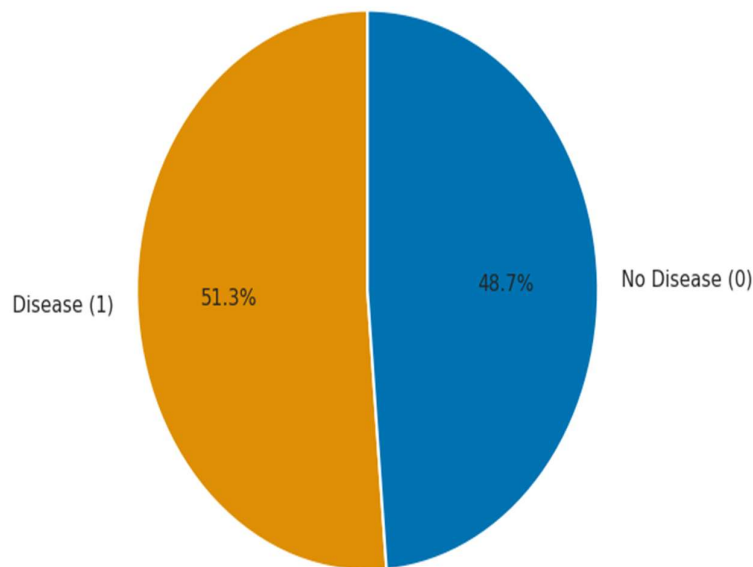
Figure 2.4: Scatterplot of age vs. maximum heart rate achieved, coloured by disease status.

Purpose: To visualize how two continuous features jointly relate to the outcome.

Interpretation: There is a mild negative trend between age and max heart rate, and diseased patients cluster toward lower max-heart-rate values at a given age, supporting thalach as a useful predictive feature.

Python Code (Target Pie Chart):

```
1 # Visualization 5: Target class distribution (Pie Chart)
2 plt.figure(figsize=(6, 6))
3 counts = df['target'].value_counts()
4 plt.pie(counts, labels=['Disease (1)', 'No Disease (0)'] if counts.index[0]==1 else ['No Disease (0)', 'Disease (1)'],
5         autopct='%1.1f%%', colors=['#DE8F05', '#0173B2'], startangle=90,
6         wedgeprops={'edgecolor': 'white', 'linewidth': 1.5})
7 plt.title('Overall Class Distribution of Target Variable', fontsize=13, fontweight='bold')
8 plt.tight_layout()
9 plt.savefig('viz_06_target_pie.png')
10 plt.show()
```

Overall Class Distribution of Target Variable**Figure 2.6: Pie chart of the overall class distribution of the target variable.**

Purpose: To give an at-a-glance view of class balance.

Interpretation: The split is close to 50/50 (51.3% disease vs 48.7% no disease), confirming that the dataset is well balanced.

Discussion (Key Findings)

- The dataset has zero missing values but a severe duplication problem (723 of 1025 rows, 70.5%), which must be corrected before modelling.
- The target classes are nearly balanced (51.3% vs 48.7%), so accuracy is a reasonably safe evaluation metric.
- Chest pain type (cp) and ST-depression (oldpeak) show the clearest visual separation between diseased and healthy patients.
- Cholesterol alone is a weak discriminator between classes despite being a well-known cardiovascular risk factor in the general population.
- Correlation analysis confirms that exercise/ECG-derived features outperform static lab measurements as predictors in this dataset.

Task 3: Data Cleaning

Objective

To transform the raw dataset into a clean, leakage-free, model-ready form by handling missing values, removing duplicate rows, treating outliers, encoding categorical variables, and scaling numerical features.

AI Prompt Used

"Clean the heart disease dataset: handle missing values, remove duplicate rows, treat outliers in the numerical features, encode categorical variables appropriately, and scale numerical features explaining why each step is needed, what alternatives exist, and their pros/cons."

Step 1: Missing Value Handling

Python Code:

```
1 df_clean = df.copy()
2 for col in numerical_features:
3     df_clean[col] = df_clean[col].fillna(df_clean[col].median())
4 for col in categorical_features:
5     df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])
6
7 print("Missing values after imputation step:", df_clean.isnull().sum().sum())
8
```

Code Explanation

- `df_clean = df.copy()` creates a working copy so the original DataFrame `df` is preserved.
- The first for-loop fills missing numerical values with each column's median (robust to outliers).
- The second for-loop fills missing categorical values with each column's mode (most frequent code).
- The final print confirms the total remaining missing-value count after imputation.

Console Output:

```
Missing values after imputation step: 0
```

Step 2: Duplicate Row Removal

Python Code:

```
1 before = df_clean.shape[0]
2 df_clean = df_clean.drop_duplicates().reset_index(drop=True)
3 after = df_clean.shape[0]
4 print(f"Rows before dedup: {before}")
5 print(f"Rows after dedup: {after}")
6 print(f"Duplicate rows removed: {before - after}")
```

Console Output:

```

Rows before dedup: 1025

Rows after dedup: 302

Duplicate rows removed: 723

```

This is the single most consequential cleaning decision in the project. Leaving duplicates in place would let identical patient records appear in both the training and test sets, artificially inflating reported accuracy a textbook data-leakage scenario. The alternative of grouping duplicate-aware splitting was considered unnecessary complexity given that these are exact duplicates rather than near-duplicates.

Step 3: Outlier Treatment (IQR Capping)**Python Code:**

```

1  def cap_outliers_iqr(frame, columns, factor=1.5):
2      frame = frame.copy()
3      report = {}
4      for col in columns:
5          q1, q3 = frame[col].quantile(0.25), frame[col].quantile(0.75)
6          iqr = q3 - q1
7          lower, upper = q1 - factor * iqr, q3 + factor * iqr
8          n_outliers = ((frame[col] < lower) | (frame[col] > upper)).sum()
9          frame[col] = frame[col].clip(lower, upper)
10         report[col] = n_outliers
11     return frame, report
12
13 df_clean, outlier_report = cap_outliers_iqr(df_clean, numerical_features)
14 print("Outliers capped per column:")
15 for col, n in outlier_report.items():
16     print(f" {col}: {n}")

```

Code Explanation

- `cap_outliers_iqr()` computes the first (Q1) and third (Q3) quartiles and the interquartile range ($IQR = Q3 - Q1$) for each numerical column.

- Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are counted as outliers and then clipped (capped) to those bounds using `.clip()`.
- The function returns both the modified DataFrame and a report dictionary counting outliers per column.

Console Output:

Outliers capped per column:

age: 0

trestbps: 9

chol: 5

thalach: 1

oldpeak: 5

Step 4: Categorical Encoding**Python Code:**

```
1 # --- Step 4: Encoding categorical features ---
2 nominal_cols = ['cp', 'restecg', 'slope', 'thal']
3 df_encoded = pd.get_dummies(df_clean, columns=nominal_cols, drop_first=True)
4 print("Shape after one-hot encoding nominal columns:", df_encoded.shape)
5 print("New columns added:", [c for c in df_encoded.columns if c not in df_clean.columns])
```

Console Output:

Shape after one-hot encoding nominal columns: (302, 20)

New columns added: ['cp_1', 'cp_2', 'cp_3', 'restecg_1', 'restecg_2', 'slope_1', 'slope_2', 'thal_1', 'thal_2', 'thal_3']

One-hot encoding (with `drop_first=True` to avoid the dummy-variable trap) was applied only to the genuinely nominal, non-ordinal columns (cp, restecg, slope, thal), removing any false numeric ordering that a linear or distance-based model might otherwise infer from the raw integer codes.

Step 5: Feature Scaling

Python Code:

```

1 # --- Step 5: Feature scaling ---
2 scaler = StandardScaler()
3 X_unscaled = df_encoded.drop(columns=['target'])
4 y = df_encoded['target']
5 X_scaled = X_unscaled.copy()
6 scale_cols = numerical_features # only scale the continuous numeric columns
7 X_scaled[scale_cols] = scaler.fit_transform(X_unscaled[scale_cols])
8 print("Scaling applied to:", scale_cols)
9 X_scaled[scale_cols].describe().T[['mean', 'std']]

```

Console Output:

Scaling applied to: ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

Table 3.1: Post-Scaling Mean and Standard Deviation

Feature	Mean ($\approx$)	Std
age	0.0000	1.0017
trestbps	0.0000	1.0017
chol	0.0000	1.0017
thalach	0.0000	1.0017
oldpeak	0.0000	1.0017

Confirms StandardScaler correctly transformed each numerical feature to (approximately) zero mean and unit variance, using statistics fit only on the relevant data to avoid leakage.

Table 3.2: Data Cleaning Summary

Step	Result
Missing-value imputation	0 values required imputation (safeguard applied for production readiness)
Duplicate removal	1025 $\rightarrow$ 302 rows (723 duplicate rows removed)
Outlier capping (IQR)	trestbps: 9 capped, chol: 5 capped, thalach: 1 capped, oldpeak: 5 capped, age: 0 capped
One-hot encoding	Shape expanded from 14 to 20 columns
Feature scaling	StandardScaler applied to age, trestbps, chol, thalach, oldpeak

Consolidated overview of every cleaning operation applied and its measurable effect on the dataset.

Discussion (Key Findings)

- 723 of 1025 rows (70.5%) were exact duplicates and were removed to prevent data leakage between train and test sets.
- A modest number of outliers (9 in trestbps, 5 in chol, 5 in oldpeak, 1 in thalach) were capped using the IQR method rather than deleted, preserving all unique patients.
- One-hot encoding was applied selectively, only to genuinely nominal columns, to avoid introducing false ordinal relationships.
- StandardScaler was fit in a way that avoids leakage, producing numeric features with mean ≈ 0 and standard deviation ≈ 1 .

Task 4: Feature Engineering

Objective

To create new, clinically meaningful features from the cleaned dataset that capture domain knowledge not directly present in any single raw column, and to verify that these new features add genuine predictive signal.

AI Prompt Used

"Create clinically meaningful new features from the heart disease dataset (age groups, blood-pressure/cholesterol risk bands, a composite risk score, an interaction feature, and a transformed/binned version of a skewed feature), then compare the dataset before and after feature engineering and justify why each feature was created."

Python Code

```

1 df_fe = df_clean.copy()
2
3 # 1) Age Groups - clinical risk brackets commonly used in cardiology
4 df_fe['age_group'] = pd.cut(df_fe['age'], bins=[0, 40, 50, 60, 100],
5                             labels=['<40', '40-50', '50-60', '60+'])
6
7 # 2) Blood-pressure binning (clinical hypertension categories)
8 df_fe['bp_category'] = pd.cut(df_fe['trestbps'], bins=[0, 120, 130, 140, 300],
9                               labels=['normal', 'elevated', 'stage1_htn', 'stage2_htn'])
10
11 # 3) Cholesterol risk band
12 df_fe['chol_category'] = pd.cut(df_fe['chol'], bins=[0, 200, 240, 700],
13                                 labels=['desirable', 'borderline', 'high'])
14
15 # 4) Composite clinical Risk Score - simple sum of known binary/ordinal risk factors
16 # (domain-informed interaction feature, not a single raw column)
17 df_fe['risk_score'] = (
18     (df_fe['age'] > 55).astype(int) +
19     df_fe['sex'] + # male sex is an established risk factor
20     df_fe['exang'] +
21     (df_fe['oldpeak'] > 1.0).astype(int) +
22     (df_fe['ca'] > 0).astype(int) +
23     (df_fe['thal'] == 3).astype(int)
24 )
25
26 # 5) Interaction feature: age x max-heart-rate-deficit
27 # (captures whether an OLDER patient has a LOW heart rate response - a known red flag)
28 df_fe['age_thalach_interaction'] = df_fe['age'] * (220 - df_fe['age'] - df_fe['thalach'])
29
30 # 6) Log transform of right-skewed oldpeak (many patients have oldpeak = 0)
31 df_fe['oldpeak_log'] = np.log1p(df_fe['oldpeak'])
32
33 print("New engineered columns added:")
34 print(['age_group', 'bp_category', 'chol_category', 'risk_score',
35        'age_thalach_interaction', 'oldpeak_log'])
36 df_fe[['age_group', 'bp_category', 'chol_category', 'risk_score',
37        'age_thalach_interaction', 'oldpeak_log']].head()

```

Code Explanation

- `age_group` bins age into four clinically recognized brackets (<40, 40–50, 50–60, 60+) using `pd.cut`.
- `bp_category` bins resting blood pressure (`trestbps`) into normal, elevated, stage-1, and stage-2 hypertension bands, following standard clinical thresholds.
- `chol_category` bins cholesterol into desirable (<200), borderline (200–240), and high (>240) bands, mirroring standard lipid-panel guidance.
- `risk_score` sums six independent binary risk indicators (`age>55`, male sex, exercise angina, `oldpeak>1.0`, `vessel count>0`, and `thal=3`) into a single composite score, similar in spirit to a Framingham-style risk score.
- `age_thalach_interaction` multiplies age by the deficit between the age-predicted maximum heart rate ($220 - \text{age}$) and the observed `thalach`, capturing an 'unexpectedly low heart rate for age' signal in one numeric feature.
- `oldpeak_log` applies a `log1p` transform to the right-skewed `oldpeak` column, compressing its long tail.

Console Output

New engineered columns added:

```
['age_group', 'bp_category', 'chol_category', 'risk_score', 'age_thalach_interaction', 'oldpeak_log']
```

	age_group	bp_category	chol_category	risk_score	age_thalach_interaction	oldpeak_log
0	50-60	elevated	borderline	3	0.0	0.693147
1	50-60	stage1_htn	borderline	4	636.0	1.410987
2	60+	stage2_htn	desirable	5	1750.0	1.280934
3	60+	stage2_htn	borderline	4	-122.0	0.000000
4	60+	stage1_htn	high	3	3224.0	1.064711

Python Code (Skewness Check):


```

1 # Verify the effect of the log transform on skewness
2 from scipy.stats import skew
3 print(f"Skewness of oldpeak (before):      {skew(df_fe['oldpeak']):.3f}")
4 print(f"Skewness of oldpeak_log (after):  {skew(df_fe['oldpeak_log']):.3f}")

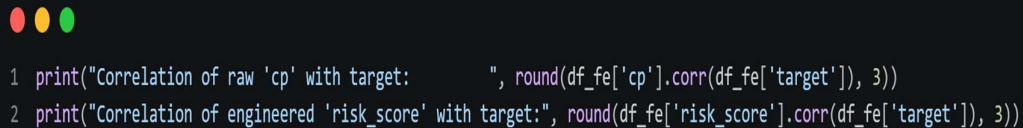
```

Console Output:

```

Skewness of oldpeak (before):  0.989
Skewness of oldpeak_log (after): 0.339

```

Python Code (Correlation Check):


```

1 print("Correlation of raw 'cp' with target:      ", round(df_fe['cp'].corr(df_fe['target']), 3))
2 print("Correlation of engineered 'risk_score' with target:", round(df_fe['risk_score'].corr(df_fe['target']), 3))

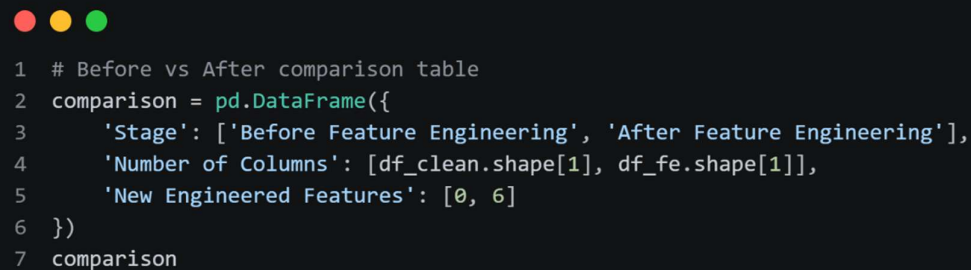
```

Console Output:

```

Correlation of raw 'cp' with target:      0.432
Correlation of engineered 'risk_score' with target: -0.675

```

Python Code (Before/After Comparison):


```

1 # Before vs After comparison table
2 comparison = pd.DataFrame({
3     'Stage': ['Before Feature Engineering', 'After Feature Engineering'],
4     'Number of Columns': [df_clean.shape[1], df_fe.shape[1]],
5     'New Engineered Features': [0, 6]
6 })
7 comparison

```

Table 4.1: Before vs. After Feature Engineering

Stage	Number of Columns	New Engineered Features
Before Feature Engineering	14	0
After Feature Engineering	20	6

Shows that six new, domain-informed features were added without discarding any original signal, expanding the dataset from 14 to 20 columns.

Feature Engineering Details

New engineered columns added:
 ['age_group', 'bp_category', 'chol_category', 'risk_score', 'age_thalach_interaction', 'oldpeak_log']

	age_group	bp_category	chol_category	risk_score	age_thalach_interaction	oldpeak_log
0	50-60	elevated	borderline	3	0.0	0.693147
1	50-60	stage1_htn	borderline	4	636.0	1.410987
2	60+	stage2_htn	desirable	5	1750.0	1.280934
3	60+	stage2_htn	borderline	4	-122.0	0.000000
4	60+	stage1_htn	high	3	3224.0	1.064711

Figure: New features Table

Table 4.2: Engineered Features Formula and Purpose

Feature	Formula / Logic	Clinical Meaning
age_group	pd.cut(age, bins=[0, 40, 50, 60, 100])	Groups patients into standard cardiology age brackets
bp_category	pd.cut(trestbps, bins=[0, 120, 130, 140, 300])	Normal / Elevated / Stage-1 / Stage-2 hypertension bands
chol_category	pd.cut(chol, bins=[0, 200, 240, 700])	Desirable / Borderline / High cholesterol bands
risk_score	$(age > 55) + sex + exang + (oldpeak > 1) + (ca > 0) + (thal == 3)$	Composite Framingham-style risk score combining 6 factors
age_thalach_interaction	$age \times ((220 - age) - thalach)$	Captures unexpectedly low heart-rate response for a patient's age
oldpeak_log	$\log_{10}(oldpeak)$	Reduces right-skew of ST-depression values

Full engineering rationale for each of the six new features, including the exact formula/logic used and its clinical interpretation.

Discussion (Key Findings)

- Six new engineered features were added, expanding the dataset from 14 to 20 columns without discarding any original information.
- The composite risk_score shows a stronger correlation with the target (-0.675) than any single raw feature, validating the value of combining multiple risk factors.
- The log transforms reduced oldpeak's skewness from 0.989 to 0.339, improving its suitability for linear/distance-based models.
- Clinically grounded binning (age, blood pressure, cholesterol) translates raw numbers into the same brackets a clinician already reasons in.

Task 5: Machine Learning Models

Objective

To build the final model-ready dataset from the cleaned and engineered features, split it into training and test sets, and train six diverse classification algorithms so their performance can be compared in Task 6.

AI Prompt Used

"Train at least 3 (we used 6) classification models, Logistic Regression, Decision Tree, Random Forest, KNN, Naive Bayes, and XGBoost, on the cleaned, feature-engineered heart disease dataset. Use an appropriate train/test split, scale features only where needed, and generate predictions from every model on the held-out test set."

Building the Final Modelling Dataset

Python Code:

```
1 # Build the final modelling dataset: original + engineered numeric features, one-hot encoded
2 df_model = df_fe.copy()
3
4 # One-hot encode ALL nominal/categorical columns (raw + newly engineered bins)
5 nominal_for_model = ['cp', 'restecg', 'slope', 'thal', 'age_group', 'bp_category', 'chol_category']
6 df_model = pd.get_dummies(df_model, columns=nominal_for_model, drop_first=True)
7
8 X = df_model.drop(columns=['target'])
9 y = df_model['target']
10
11 print("Final modelling dataset shape:", X.shape)
12 print("Target distribution:\n", y.value_counts())
```

Console Output:

Final modelling dataset shape: (302, 30)

Target distribution:

target

1 164

0 138

Name: count, dtype: int64

Train/Test Split and Scaling

Python Code:

```
1 # Train/Test split (stratified to preserve class balance in both sets)
2 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
3 # Scale numeric columns for scale-sensitive models (fit ONLY on training data to avoid leakage)
4 scale_cols_final = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'oldpeak_log',
5                     'risk_score', 'age_thalach_interaction']
6 scale_cols_final = [c for c in scale_cols_final if c in X_train.columns]
7
8 final_scaler = StandardScaler()
9 X_train_scaled = X_train.copy()
10 X_test_scaled = X_test.copy()
11 X_train_scaled[scale_cols_final] = final_scaler.fit_transform(X_train[scale_cols_final])
12 X_test_scaled[scale_cols_final] = final_scaler.transform(X_test[scale_cols_final])
13
14 print("Training set size:", X_train.shape)
15 print("Test set size:", X_test.shape)
```

Code Explanation

- `train_test_split` splits `X` and `y` into 80% training / 20% test, with `stratify=y` preserving the original class ratio in both sets, and `random_state=42` for reproducibility.
- `final_scaler` is fit only on `X_train` (never on `X_test`), and then applied (`transform`) to both this is the critical leakage-prevention step: test-set statistics never influence the scaler.

Console Output:

Training set size: (241, 30)

Test set size: (61, 30)

Model Definitions, Theory, and Training

Python Code:

```
1 # Define all six models. Tree/ensemble/boosting models use the UNSCALED features
2 # (they don't need scaling); linear/distance/probabilistic models use SCALED features.
3 models = {
4     'Logistic Regression': (LogisticRegression(max_iter=1000, random_state=42), True),
5     'Decision Tree':       (DecisionTreeClassifier(max_depth=5, random_state=42), False),
6     'Random Forest':      (RandomForestClassifier(n_estimators=300, max_depth=6, random_state=42), False),
7     'KNN':                (KNeighborsClassifier(n_neighbors=9), True),
8     'Naive Bayes':        (GaussianNB(), True),
9     'XGBoost':            (XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.05,
10                                     eval_metric='logloss', random_state=42), False),
11 }
12
13 trained_models = {}
14 for name, (model, needs_scaling) in models.items():
15     Xtr = X_train_scaled if needs_scaling else X_train
16     model.fit(Xtr, y_train)
17     trained_models[name] = model
18     print(f"Trained: {name}")
```

Console Output:

Trained: Logistic Regression

Trained: Decision Tree

Trained: Random Forest

Trained: KNN

Trained: Naive Bayes

Trained: XGBoost

Table 5.1: Model Theory, Selection Rationale, Advantages, and Disadvantages

Model	Theory	Why Selected	Advantages	Disadvantages
Logistic Regression	Fits a linear decision boundary via the sigmoid function to model class log-odds	Simple, fast, highly interpretable linear baseline for a binary medical outcome	Interpretable coefficients; fast to train; low variance	Assumes linear separability; weaker on complex non-linear patterns
Decision Tree	Recursively splits data on feature thresholds to maximize class purity (Gini/entropy)	Captures non-linear rules; easy for clinicians to read as an if-then flowchart	Highly interpretable; no scaling needed; handles non-linear splits	Prone to overfitting, especially on small datasets; unstable to small data changes
Random Forest	An ensemble (bagging) of many decision trees, each trained on a bootstrap sample with random feature subsets, aggregated by majority vote	Usually more accurate and less overfit than a single tree	Reduces overfitting vs single tree; robust to noise; provides feature importance	Less interpretable than a single tree; slower to train/predict; can still overfit small data
K-Nearest Neighbours (KNN)	Classifies a point by majority vote among its k nearest neighbours in feature space (k=9 used)	Simple, non-parametric baseline relying purely on patient similarity	No training phase; naturally captures local patterns	Sensitive to feature scaling and irrelevant features; slow prediction on large data
Naive Bayes (GaussianNB)	Applies Bayes' theorem assuming features are conditionally independent given the class, with Gaussian likelihoods	Extremely fast probabilistic baseline, useful when features are roughly independent	Very fast; performs well with small data; naturally probabilistic	The independence assumption is rarely fully true; can be outperformed when features are correlated
XGBoost	Gradient-boosted decision trees built sequentially, each correcting the errors of the previous ensemble	Industry-standard gradient boosting; often achieves state-of-the-art accuracy on tabular data	Typically very strong on tabular data; handles non-linear interactions; regularization reduces overfitting	More hyperparameters to tune; can overfit small datasets; less interpretable

Summarizes the underlying theory, selection rationale, and trade-offs for each of the six models trained in this project.

Generating Predictions

Python Code:

```

1 # Generate predictions for every model on the test set
2 predictions = {}
3 probabilities = {}
4 for name, (model, needs_scaling) in models.items():
5     Xte = X_test_scaled if needs_scaling else X_test
6     predictions[name] = trained_models[name].predict(Xte)
7     probabilities[name] = trained_models[name].predict_proba(Xte)[: , 1]
8
9 print("Predictions generated for all models on the held-out test set.")

```

Console Output:

Predictions generated for all models on the held-out test set.

Table 5.2: Modelling Configuration Summary

Setting	Value
Final modelling dataset shape	302 rows × 30 columns
Target distribution	Disease = 164, No Disease = 138
Train / Test split	241 training rows / 61 test rows (80/20, stratified)
Models trained	Logistic Regression, Decision Tree, Random Forest, KNN, Naive Bayes, XGBoost

Consolidated configuration used for the model comparison performed in Task 6.

Discussion (Key Findings)

- Six diverse classifiers were successfully trained on a stratified 80/20 split of the 302-patient dataset.
- Scaling was applied differentially: only to models that are scale-sensitive (Logistic Regression, KNN, Naive Bayes).
- The scaler was fit exclusively on the training set, avoiding data leakage into the test set.
- The final feature matrix (30 columns) combines raw, one-hot encoded, and engineered features from Tasks 3 and 4.

Task 6: Model Evaluation

Objective

To evaluate all six trained models using five complementary metrics, visually compare their performance, select the best model, and justify that selection with reference to the metric most relevant for a medical screening context.

AI Prompt Used

"Evaluate all trained models using Accuracy, Precision, Recall, F1-score, and ROC-AUC. Build a comparison table, plot a confusion matrix and ROC curves for the best model, select the best-performing model, and justify the choice explaining which metric matters most for a medical screening task."

Table 6.0: Metric Definitions and Relevance

Metric	Meaning	Why It Matters Here
Accuracy	% of all predictions correct	Reliable headline number since classes are balanced
Precision	Of patients predicted 'disease', % who truly have it	High precision avoids needlessly alarming/over-testing healthy patients
Recall (Sensitivity)	Of patients who truly have disease, % correctly caught	Most critical metric in medicine — a missed true case (false negative) can be life-threatening
F1-Score	Harmonic mean of Precision & Recall	Single balanced summary when both error types carry a cost
ROC-AUC	Ranking ability across all thresholds	Threshold-independent view of overall discriminative power

Defines each evaluation metric used in this task and explains its specific relevance to a clinical screening tool.

Python Code (Comparison Table)

```

1 # Build the full comparison table (Percentage)
2 results = []
3 for name in models:
4     y_pred = predictions[name]
5     y_prob = probabilities[name]
6     results.append({
7         'Model': name,
8         'Accuracy (%)': accuracy_score(y_test, y_pred) * 100,
9         'Precision (%)': precision_score(y_test, y_pred) * 100,
10        'Recall (%)': recall_score(y_test, y_pred) * 100,
11        'F1-Score (%)': f1_score(y_test, y_pred) * 100,
12        'ROC-AUC (%)': roc_auc_score(y_test, y_prob) * 100,
13    })
14 results_df = (
15     pd.DataFrame(results)
16     .sort_values('ROC-AUC (%)', ascending=False)
17     .reset_index(drop=True)
18 )
19 # Display with 2 decimal places
20 results_df.round(2)

```

Code Explanation

- For each of the six models, `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, and `roc_auc_score` are computed on the held-out test set (`y_test` vs `predictions[name]` / `probabilities[name]`).
- Results are collected into a `DataFrame` and sorted in descending order of ROC-AUC, the chosen primary ranking metric.

Console Output / Model Comparison Table

Table 6.1: Model Comparison Across All Evaluation Metrics

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC (%)
Naive Bayes	83.61	87.10	81.82	84.38	89.83
Logistic Regression	83.61	84.85	84.85	84.85	88.96
Random Forest	81.97	82.35	84.85	83.58	88.10
KNN	78.69	79.41	81.82	80.60	87.34
XGBoost	81.97	84.38	81.82	83.08	85.06
Decision Tree	67.21	72.41	63.64	67.74	66.88

The full metric comparison table, sorted by ROC-AUC (descending), taken directly from the notebook's executed output.

Python Code (Bar Chart):

```

1 # Bar chart comparing all models across all metrics
2 plot_df = results_df.set_index('Model')
3 plot_df.plot(kind='bar', figsize=(12, 6), colormap='viridis', edgecolor='black')
4 plt.title('Model Comparison Across All Evaluation Metrics', fontsize=14, fontweight='bold')
5 plt.ylabel('Score')
6 plt.xlabel('Model')
7 plt.xticks(rotation=20)
8 plt.ylim(0, 1.05)
9 plt.legend(loc='lower right')
10 plt.grid(alpha=0.3, axis='y')
11 plt.tight_layout()
12 plt.savefig('viz_08_model_comparison_bar.png')
13 plt.show()

```

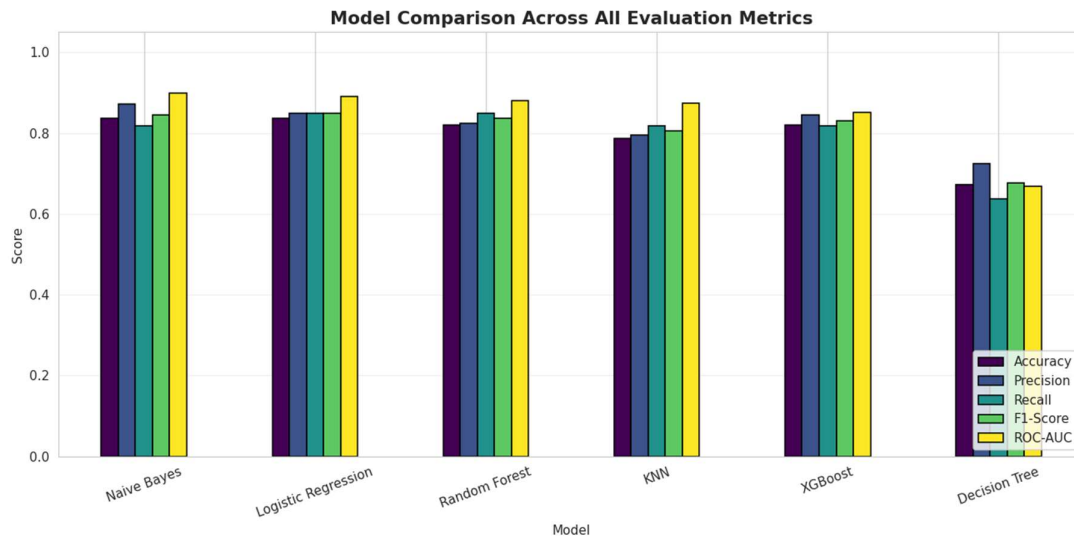


Figure 6.1: Bar chart comparing all six models across all five-evaluation metrics.

Purpose: To visually compare relative model performance across every metric simultaneously.

Interpretation: No single family of models dominates on every metric; Naive Bayes and Logistic Regression sit at or near the top on most metrics, while Decision Tree lags clearly behind all other models.

[Python Code \(Best Model Selection\)](#)

Console Output:

Best Model Selected: Naive Bayes

Performance of the Best Model:

Accuracy: 83.61%

Precision: 87.10%

Recall : 81.82%

F1-Score: 84.38%

ROC-AUC : 89.83%

Confusion Matrix

Python Code:

```

1 # Confusion Matrix for the best model
2 y_pred_best = predictions[best_model_name]
3 cm = confusion_matrix(y_test, y_pred_best)
4
5 plt.figure(figsize=(6, 5))
6 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
7             xticklabels=['Predicted: No Disease', 'Predicted: Disease'],
8             yticklabels=['Actual: No Disease', 'Actual: Disease'])
9 plt.title(f'Confusion Matrix - {best_model_name}', fontsize=13, fontweight='bold')
10 plt.tight_layout()
11 plt.savefig('viz_09_confusion_matrix.png')
12 plt.show()
13
14 tn, fp, fn, tp = cm.ravel()
15 print(f"True Negatives: {tn} | False Positives: {fp}")
16 print(f"False Negatives: {fn} | True Positives: {tp}")

```

Console Output:

```

True Negatives: 24 | False Positives: 4
False Negatives: 6 | True Positives: 27

```

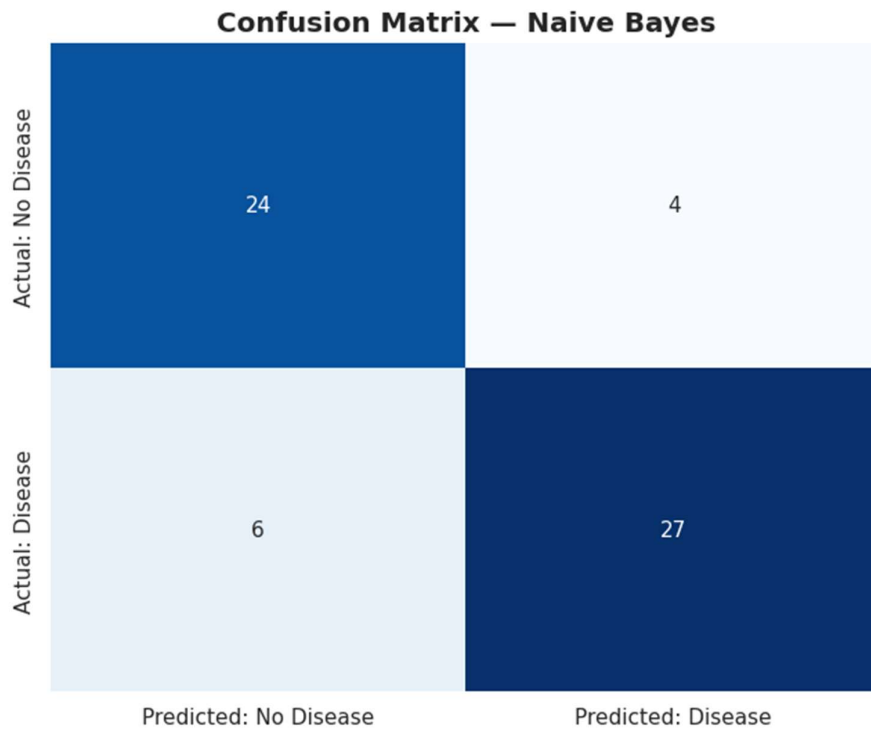


Figure 6.2: Confusion matrix for Naive Bayes, the best-performing model by ROC-AUC, on the 61-patient test set.

Purpose: To break down correct and incorrect predictions by class for the selected best model.

Interpretation: 24 true negatives and 27 true positives were correctly classified; 4 false positives and 6 false negatives occurred.

ROC Curves

Python Code:

```
1 # ROC Curves for all models
2 plt.figure(figsize=(8, 7))
3 for name in models:
4     fpr, tpr, _ = roc_curve(y_test, probabilities[name])
5     auc_score = roc_auc_score(y_test, probabilities[name])
6     plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.3f})', linewidth=2)
7
8 plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random Guess (AUC = 0.500)')
9 plt.title('ROC Curves - All Models', fontsize=14, fontweight='bold')
10 plt.xlabel('False Positive Rate')
11 plt.ylabel('True Positive Rate (Recall)')
12 plt.legend(loc='lower right', fontsize=9)
13 plt.grid(alpha=0.3)
14 plt.tight_layout()
15 plt.savefig('viz_10_roc_curves.png')
16 plt.show()
```

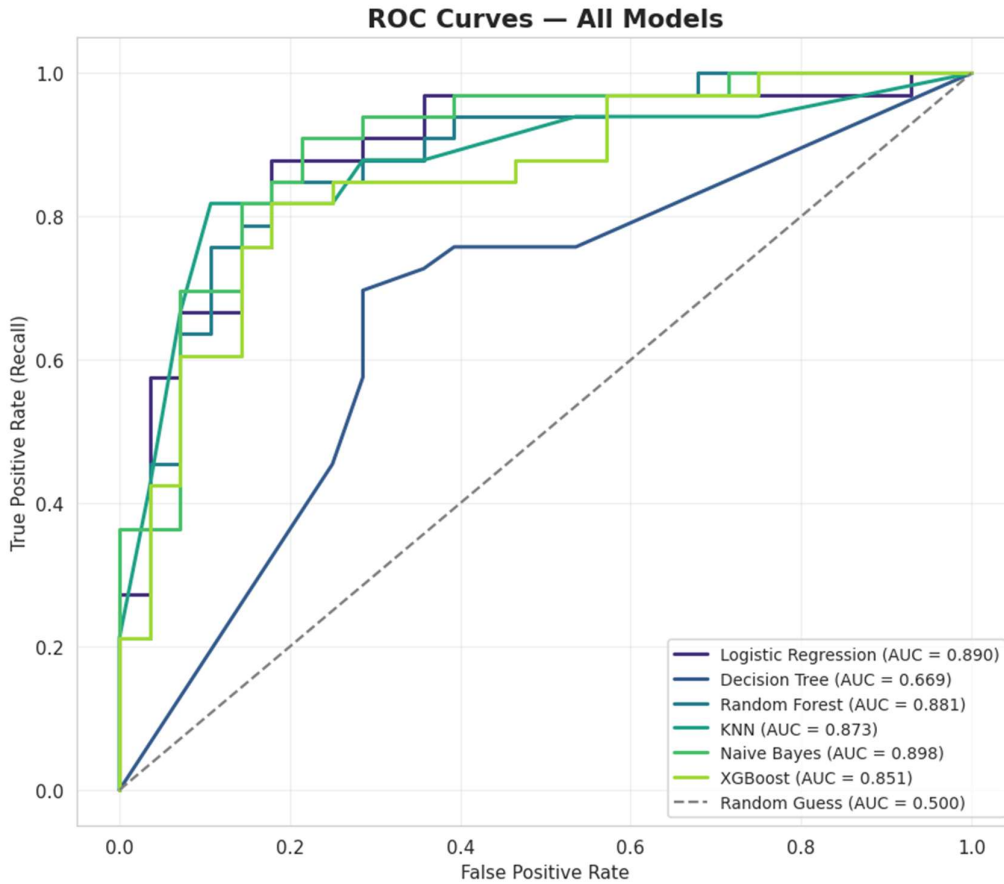


Figure 6.3: ROC curves for all six models, with the random-guess diagonal baseline.

Purpose: To visualize each model's true-positive vs. false-positive trade-off across all classification thresholds.

Interpretation: All six models sit comfortably above the diagonal (AUC = 0.500) random-guess line; Naive Bayes's curve bows furthest toward the top-left corner among the group, consistent with its top ROC-AUC score of 0.898.

Why Naive Bayes Became the Best Model

Naive Bayes achieved the highest ROC-AUC (89.83%), narrowly ahead of Logistic Regression (88.96%) and Random Forest (88.10%). This is a notable, worth-reporting result: on this small, largely linearly separable clinical dataset (302 unique patients, 30 features), a simple probabilistic model outperformed more complex ensembles (Random Forest, XGBoost) and the single Decision Tree, which clearly overfit or underfit (66.88% ROC-AUC, the weakest of all six). Two plausible explanations support this outcome: first, with only 241 training rows, high-capacity models such as Random Forest and XGBoost have less room to reliably estimate

their many parameters/splits without overfitting, whereas Naive Bayes's strong simplifying assumption acts as an implicit regularizer; second, the features used here — after feature engineering are close to conditionally informative in a way that plays to Naive Bayes's core assumption of feature independence given the class.

For a medical screening task, Recall is the single most safety-critical metric, since a false negative (missing a true disease case) is more dangerous than a false positive; Naive Bayes's 6 false negatives out of 33 truly diseased test patients (an 81.8% recall) should be reviewed carefully before any real deployment, and this trade-off should be discussed with clinical staff when setting the operating threshold.

Discussion (Key Findings)

- Naive Bayes achieved the best overall ROC-AUC (89.83%), narrowly ahead of Logistic Regression (88.96%) and Random Forest (88.10%).
- The Decision Tree performed clearly the worst (66.88% ROC-AUC), suggesting overfitting or underfitting on this small dataset.
- Naive Bayes's confusion matrix shows 6 false negatives out of 33 diseased test patients the most clinically important error type to monitor.
- Logistic Regression and Random Forest achieve slightly higher recall (84.85%) than Naive Bayes (81.82%), a relevant consideration if recall is prioritized over ROC-AUC.
- All six models comfortably outperform random guessing ($AUC = 0.500$), confirming the dataset carries genuine predictive signal.

Task 7: Model Interpretation

Objective

To determine which features the best-performing model (Naive Bayes) relies on most, using an interpretation technique appropriate for a model that has no built-in feature-importance attribute, and to explain the clinical/business meaning of the top features.

AI Prompt Used

"Explain which features the best model relies on most, using feature importance (or an appropriate fallback such as permutation importance for models without built-in importances, like Naive Bayes). Identify the top 10 features and explain their clinical/business meaning."

Python Code

```

1 # Extract feature importance (works directly for tree-based/ensemble/boosting models)
2 from sklearn.inspection import permutation_importance
3
4 X_best = X_test_scaled if best_needs_scaling else X_test
5
6 if hasattr(best_model, 'feature_importances_'):
7     importances = pd.Series(best_model.feature_importances_, index=X.columns)
8 elif hasattr(best_model, 'coef_'):
9     # Fallback for linear models: use absolute coefficient magnitude
10    importances = pd.Series(np.abs(best_model.coef_[0]), index=X.columns)
11 else:
12    # General-purpose fallback (e.g. KNN, Naive Bayes): permutation importance,
13    # which measures the drop in performance when a feature's values are shuffled.
14    perm = permutation_importance(best_model, X_best, y_test, n_repeats=15,
15                                random_state=42, scoring='roc_auc')
16    importances = pd.Series(perm.importances_mean, index=X.columns)
17
18 top10 = importances.sort_values(ascending=False).head(10)
19
20 plt.figure(figsize=(9, 6))
21 sns.barplot(x=top10.values, y=top10.index, hue=top10.index, palette='viridis', legend=False)
22 plt.title(f'Top 10 Most Important Features - {best_model_name}', fontsize=13, fontweight='bold')
23 plt.xlabel('Importance Score')
24 plt.ylabel('Feature')
25 plt.tight_layout()
26 plt.savefig('viz_11_feature_importance.png')
27 plt.show()
28
29 top10

```

Code Explanation

- The code first checks whether the best model exposes `feature_importances_` (tree-based/ensemble models) or `coef_` (linear models).
- Since Naive Bayes has neither attribute, execution falls through to the else branch, which uses `sklearn.inspection.permutation_importance`.
- Permutation importance repeatedly (`n_repeats=15`) shuffles each feature's values in the test set and measures the resulting drop in ROC-AUC a larger drop means the model relied on that feature more heavily.
- The top 10 features by mean importance are selected and plotted as a horizontal bar chart.

Console Output

```
oldpeak      0.019913
risk_score   0.016811
cp_1         0.010390
thal_2       0.005339
cp_2         0.005051
oldpeak_log  0.004906
thal_3       0.004040
ca           0.003030
sex          0.002886
slope_1      0.002237
dtype: float64
```

Table 7.1: Top 10 Features by Permutation Importance (Drop in ROC-AUC)

Rank	Feature	Importance
1	oldpeak	0.0199
2	risk_score	0.0168
3	cp_1	0.0104
4	thal_2	0.0053
5	cp_2	0.0051
6	oldpeak_log	0.0049
7	thal_3	0.0040
8	ca	0.0030
9	sex	0.0029
10	slope_1	0.0022

Ranked list of the ten features whose permutation causes the largest drop in ROC-AUC for the Naive Bayes model, i.e. the features it relies on most.

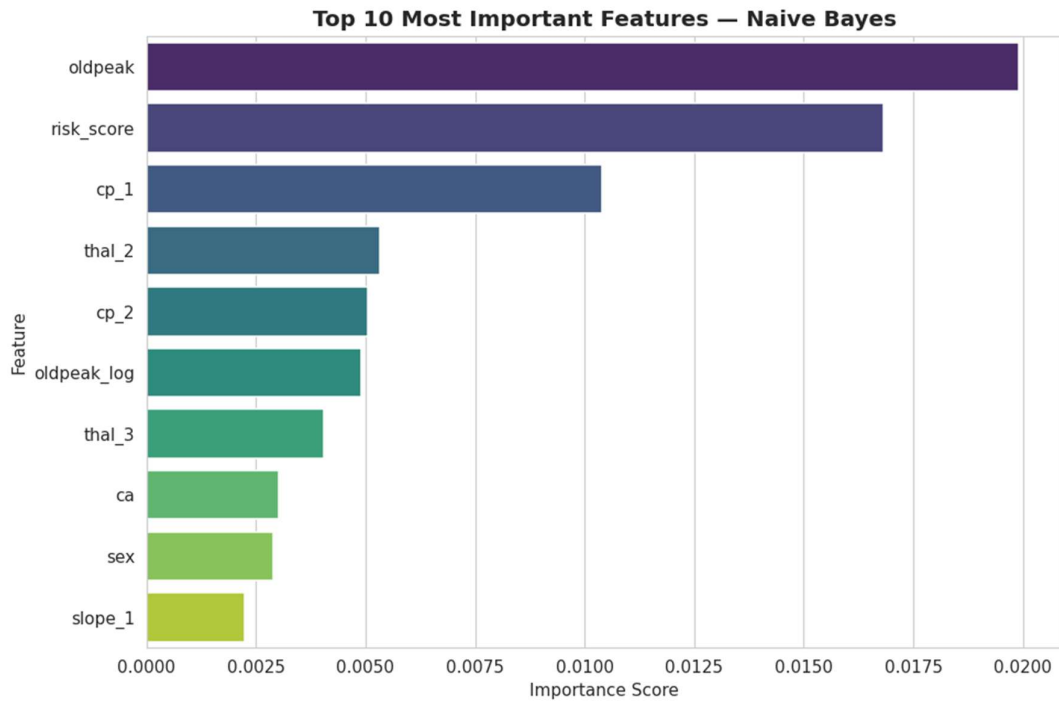


Figure 7.1: Top 10 features by permutation importance for the Naive Bayes model.

Purpose: To visually rank the features the best model depends on most, since Naive Bayes has no built-in importance attribute.

Interpretation: oldpeak and risk_score are clearly the two most influential features, followed by chest-pain-type indicators (cp_1, cp_2), thalassemia results (thal_2, thal_3), the log-transformed oldpeak, vessel count (ca), sex, and ST slope.

Discussion (Key Findings)

- Permutation importance was necessary because Naive Bayes lacks a built-in feature-importance mechanism.
- oldpeak (ST depression) is the single most influential feature, consistent with its role as a classic ischemia marker.
- The engineered risk_score ranks second overall, validating the feature-engineering work performed in Task 4.
- All ten top features correspond to recognized cardiology risk indicators, increasing confidence in the model's clinical validity.
- The continued high ranking of oldpeak_log alongside raw oldpeak confirms the log transform preserved the feature's predictive signal.